

Test Method Execution Basics

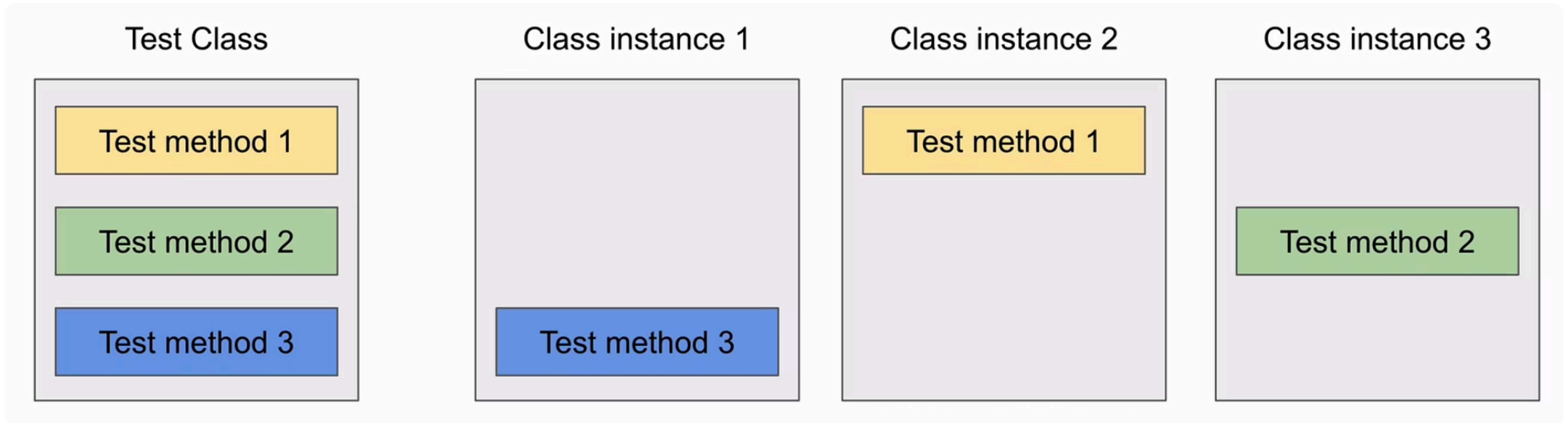
Understanding how **JUnit** manages **test lifecycle** and **execution order**

 **by Rodrigo Martins Pagliares**

Last update: 11/26/2025



Test Method Execution Basics



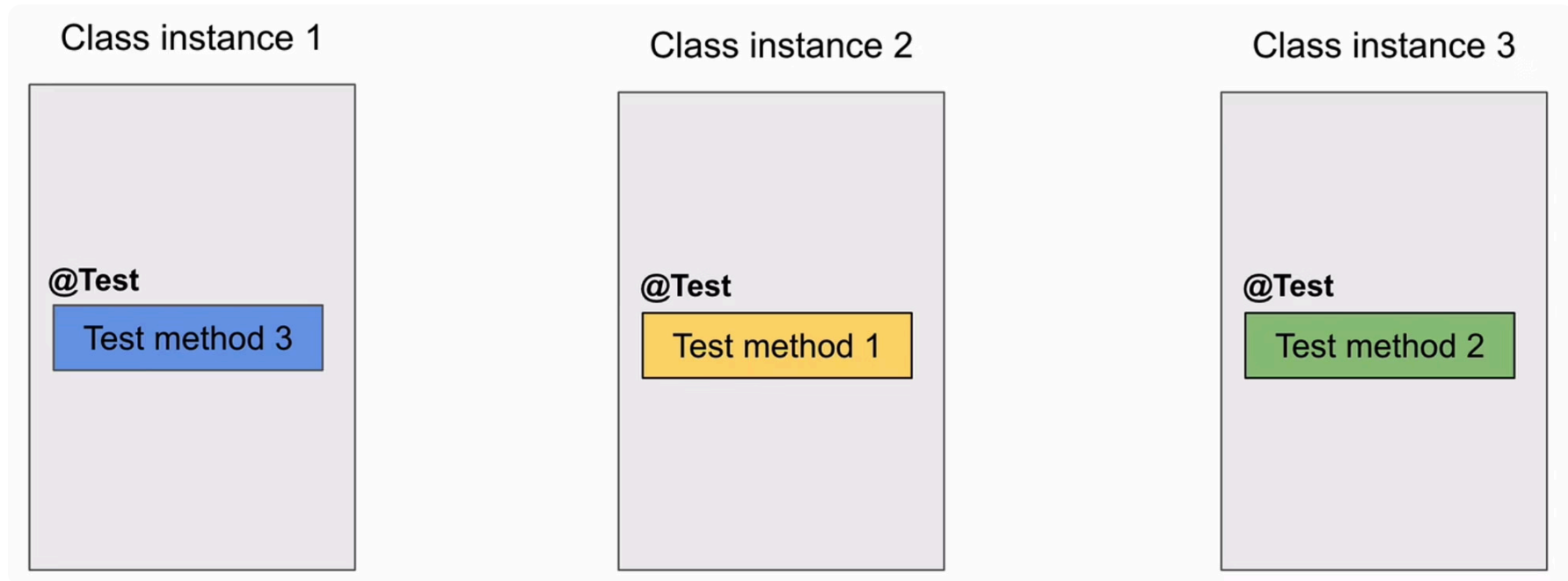
New Instance Per Test

Each test method runs with a **new instance** of the **test class** by default

Best Practice

Assume tests may run in **any order**

Test Method Execution Basics



No Guaranteed Order

JUnit does **not** guarantee a **predictable execution order** for tests

Lifecycle Management

Ensures predictable **setup** and **cleanup** where needed

@BeforeAll and @AfterAll



@BeforeAll

Runs **once before all tests** in the class

- Used to initialize expensive resources (e.g., create database)

Requires method to be **static** unless using a specialized test instance mode

@AfterAll

Runs **once after all tests finish**

- Used for cleanup such as dropping databases or closing servers

@BeforeAll

@BeforeAll

setup()

Class instance 1

@Test

Test method 3

Class instance 2

@Test

Test method 1

Class instance 3

@Test

Test method 2

@BeforeAll

Runs **once before all tests** in the class

- Used to initialize expensive resources (e.g., create database)

Requires method to be **static** unless using a specialized test instance mode

@AfterAll

@BeforeAll

setup()

Class instance 1

@Test

Test method 3

Class instance 2

@Test

Test method 1

Class instance 3

@Test

Test method 2

@AfterAll

cleanup()

@AfterAll

Runs **once after all tests finish**

- Used for cleanup such as dropping databases or closing servers

@BeforeEach and @AfterEach



@BeforeEach

Runs **before every test method**

- Ideal for creating objects reused by each test
- Prevents copy-paste initialization across tests

@AfterEach

Runs **after every test method**

- Useful for cleanup, such as closing connections or resetting state

Ensures each test runs in a clean environment

@BeforeEach

@BeforeAll

setup()

Class instance 1

@BeforeEach

beforeEach()

@Test

Test method 3

@AfterEach

afterEach()

Class instance 2

@BeforeEach

beforeEach()

@Test

Test method 1

@AfterEach

afterEach()

Class instance 3

@BeforeEach

beforeEach()

@Test

Test method 2

@AfterEach

afterEach()

@AfterAll

cleanup()

@BeforeEach

Runs **before every test method**

- Ideal for creating objects reused by each test
- Prevents copy-paste initialization across tests

@AfterEach

@BeforeAll

setup()

Class instance 1

@BeforeEach

beforeEach()

@Test

Test method 3

@AfterEach

afterEach()

Class instance 2

@BeforeEach

beforeEach()

@Test

Test method 1

@AfterEach

afterEach()

Class instance 3

@BeforeEach

beforeEach()

@Test

Test method 2

@AfterEach

afterEach()

@AfterAll

cleanup()

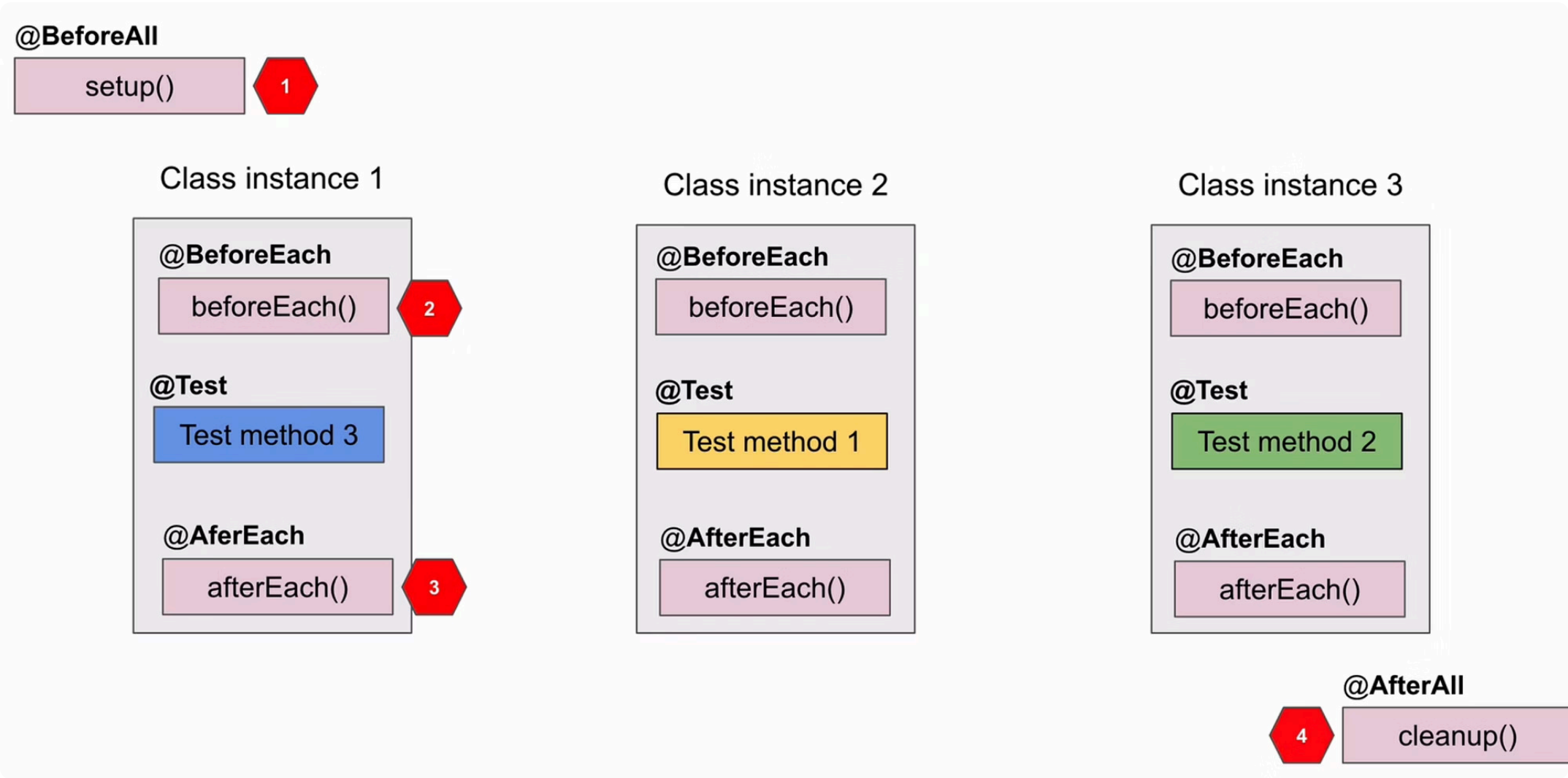
@AfterEach

Runs **after every test method**

- Useful for cleanup, such as closing connections or resetting state

Ensures each test runs in a clean environment

Lifecycle Summary



01

@BeforeAll

One-time setup for entire test class

02

@BeforeEach

Setup for each individual test

03

@AfterEach

Cleanup after each test

04

@AfterAll

Final cleanup after all tests have run

Together, they enforce predictable test behavior

Conclusion

- Test **lifecycle methods** improve **readability** and **reduce duplicated setup logic**
 - They **support isolation, maintainability, and reliability** of **test suites**
 - Using the correct lifecycle hook helps **keep tests fast** and **deterministic**
 - These **patterns** scale well for both **small** and **large applications**
-

Open Floor Discussion

- Which lifecycle hook do you think you'll use most often?
- Would your current tests benefit from `@BeforeEach` refactoring?
- Are there expensive operations you could move to `@BeforeAll`?

